LLDD BE - Job 3 ImportImpactCompetitor

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. บทนำ

1.1 วัตถุประสงค์

นำเข้าร้านคู่แข่งจาก ALLMAP: นำข้อมูลร้านคู่แข่งรายงวดจาก ไฟล์ Impact_Competitor_YYYYMMDD.txt บน EAI S3 (interface/in/ALM/ · โฟลเดอร์เดียวกับไฟล์ของ Job 2 · เปลี่ยนจากวิว COMPETITOR_IMPACT_VIEW ของ SQL Server เมื่อ 2026-09-30) เข้า sgi_fgi_impact_competitors ทีละ 10,000 แถว กันซ้ำระดับงวด (ถ้างวดมีข้อมูลแล้วจะข้ามทั้งงวด ไม่มี upsert)

1.2 ขอบเขตงาน

- Main class/script: th.co.gosoft.fgi.main.ImportImpactCompetitor / /appstore/SPS/FGI/schedule/FGI_ImportCompetitor.sh
- Phase: A
- Output: sgi_fgi_impact_competitors
- Estimate: 9 ชั่วโมง
- Argument รับผ่าน `INPUT` (JSON) — local ใช้ env `JOB_NAME/INPUT` · AWS Batch ใช้ `argv[3]/argv[2]` · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอควบคุม (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี `@Cron`) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึกลง `integration_log` โดย `main.ts` อัตโนมัติ + structured log `BATCH_START/BATCH_END` พร้อม `runId`

1.3 ข้อมูลส่งมอบ

รายการ	รายละเอียด
Track	BE
Estimate	12 ชั่วโมง = implementation 9 + unit test 3 (30%)
Owner	Aphiwit <Bank> Khammoon
Legacy class เดิม	`ImportImpactCompetitor` (`fcsJar/src/th/co/gosoft/fgi/main/ImportImpactCompetitor.java`)
Target repository	`SBP/srm-sps-spsap-sop-sgi-batch` (NestJS 11 + TypeORM · schema `sps_store` · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน `src/main.ts` แล้วรับ argument ผ่าน `JOB_NAME/INPUT` (local) หรือ `argv[3]/argv[2]` (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ `@Cron` · ดู `srm sps spsap sop sgi batch`

1.4 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success,data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-2-ImportImpactStore.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Program Specification

2.1 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-import-impact-competitor` ใน `src/main.ts` ของ `SBP/srm-sps-spsap-sop-sgi-batch` โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรัน	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	env `JOB_NAME`	env `INPUT` (JSON string)	`JOB_NAME=sgi-import-impact-competitor INPUT='{"s3Key":"interface/in/ALM/Impact_Competitor_20260929.txt"}' npm run start`
AWS Batch (ตารางเวลาจริง)	`process.argv[3]`	`process.argv[2]` (JSON string)	`node dist/main.js '{"s3Key":"interface/in/ALM/Impact_Competitor_20260929.txt"}' sgi-import-impact-competitor`

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron`/`@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: `args[0]` = `YYYY|MM` — ต้องมี 2 ส่วนพอดี (ว่าง/เกิน = `NullPointerException`) · validate ด้วย `isDateValid(yyyy/MM)` · ไม่ส่ง = งวดเดือนก่อนหน้า
ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่
ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างน้อย — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
`year`	number	งวดเดือนก่อนหน้า	ค.ศ. 4 หลัก · ต้องส่งคู่กับ `month` เสมอ	`paramsArr[0]`
`month`	number	งวดเดือนก่อนหน้า	1-12 · ส่งมาตัวเดียวโดยไม่มีอีกตัว = `INVALID_JOB_INPUT`	`paramsArr[1]`
`s3Key`	string	`null` = หยิบไฟล์ล่าสุด	key ที่ EventBridge ส่งมา · ต้องอยู่ใต้ `interface/in/ALM/` และขึ้นต้นด้วย `Impact_Competitor_`	ของใหม่ — มติ 2026-10-01 (trigger ด้วย S3 event)
`dryRun`	boolean	`false`	`true` = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
`limit`	number	`null`	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกาการ validate ที่ทุก job ต้องทำเหมือนกัน — parse `INPUT` ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เงียบ ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ล้ม

- • มติ 2026-10-01 — ไม่ตั้งเวลาแล้ว · ทำงานเมื่อไฟล์ `Impact_Competitor_*.txt` มาถึง `interface/in/ALM/` (rule แยกจากของ Job 2 ตามชื่อไฟล์) · ต้องรอแถวแม่จาก Job 2 — ร้านที่ไม่มีแถวแม่ของงวดถูกข้าม + นับ + alert
- legacy โยน exception ทันทีเมื่อ argument ผิดรูป — job ใหม่ต้องคงพฤติกรรม fail-fast ไม่ใช่ fallback เงียบ ๆ ไปงวดก่อนหน้า

2.2 Input / Output

Stage	Contract for implementation
Input	Period year/month and competitor impact data from ALLMAP Impact_Competitor_*.txt read from EAI S3 (interface/in/ALM/).
Progress	validate period, skip when period already exists, query competitor view, insert in chunks inside a transaction, send status mail.
Output	FGI_IMPACT_COMPETITOR rows for the target period; run status is success/no-data/failed with inserted-count reconciliation.

2.3 ขั้นตอนการทำงาน

Step	Description
1	เริ่ม
2	เป็นงวดใหม่ (ยังไม่เคยนำเข้า)? | No: ข้ามทั้งงวดแล้วจบด้วยสถานะ SKIPPED — ไม่ query ไม่ insert (กันซ้ำระดับงวด (Errata E15) · บันทึก existingRowCount + skippedPeriod=true · alert เมื่อ existingRowCount ไม่สมเหตุสมผล)
3	อ่านไฟล์ Impact_Competitor จาก EAI S3 แล้ว DISTINCT 11 คอลัมน์ในโค้ด (interface/in/ALM/ · คัดด้วย prefix ชื่อไฟล์ จึงไม่หยิบไฟล์ของ Job 2)
4	พบข้อมูลต้นทาง? | No: จบการทำงาน
5	insert ทีละ 10,000 แถว (ผูก impact_process_id) (map COMPET_ID → competitor_store_code (ว่าง → NULL) · map ชื่อแบรนด์ → brand_code (map ไม่ได้ = NULL + นับ unmapped) · NAMT → name_th · BRANCHT → branch_th (NAMT/BRANCHT เป็นชื่อคอลัมน์จริงของวิว ALLMAP))
6	reconcile: inserted + rejected + deduplicated = จำนวนต้นทาง? | No: throw ภายใน transaction เพื่อให้ rollback ทั้งงวด (▲ ต้องตรวจ ภายใน transaction ก่อน commit — ถ้าตรวจหลัง commit ข้อความ Rollback จะไม่เป็นความจริง)
7	Commit / จบ

2.4 เงื่อนไขตัดสิน (Decision Rules) — ตัดสินจากอะไร

Job 3 ตัดสินเรื่องเดียวแต่พลาดง่าย: งวดนี้เคยนำเข้าคู่แข่งไปแล้วหรือยัง — ▲ ระบบเดิม ข้ามทั้งงวดแล้วรายงานว่าสำเร็จ ไม่ได้ล้างแล้วนำเข้าใหม่ และไม่ได้ merge ทีละแถว (ยืนยันแล้วว่า `fcsJar/` ไม่มีคำสั่ง `DELETE FROM FGI_IMPACT_COMPETITOR` เลยแม้แต่จุดเดียว)

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
งวดนี้มีข้อมูลคู่แข่งอยู่แล้วหรือไม่	`sgi_fgi_impact_competitors` — `impact_process_id` · `period_key` (CHAR(7) `'YYYY-MM'`)	มีแถวของ `period_key` = งวดที่ขอ อยู่แล้ว → ถือว่า "งวดนี้นำเข้าแล้ว"	ข้ามทั้งงวดทันที ไม่ insert อะไรเลย แล้วจบด้วยสถานะสำเร็จ (`ImportController.importImpactCompetitor` บรรทัด 521-527) — ไม่ใช่ล้างแล้วนำเข้าใหม่ และไม่ใช่ upsert · ● จุดเสี่ยง: ถ้ารอบก่อนล้มกลางคันจนมีข้อมูลค้างไม่ครบ การรันซ้ำจะไม่ช่วยอะไร ข้อมูลขาดถาวรโดยไม่มีใครรู้ — ระบบใหม่ต้องเทียบจำนวนกับต้นทางแล้ว alert เมื่อไม่ตรง (ดู JOB 03 ImportImpactCompetitor อธิบายละเอียด หัวข้อ 3 และ 10)
ร้านคู่แข่งรายนี้อ้างอิงได้หรือไม่	`sgi_competitors.competitor_code` (master 12 รหัส `01`-`12`)	▲ `COMPET_ID` ที่วิวส่งมาคือ *รหัสสาขา* ไม่ใช่รหัสแบรนด์ (ไฟล์จริง BPM06003O มี 222 รหัสจาก 7 แบรนด์) จึงเก็บลง `competitor_store_code` ที่ไม่มี FK · ส่วน `brand_code` ได้จากการ map ชื่อแบรนด์เข้า master 11 รายการ	ต้อง reject รายแถวพร้อมเก็บ reason ไม่ใช่ล้มทั้งงวด (นับเข้า `rejected` ใน metrics)

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
แถวซ้ำในวิวต้นทาง	ไฟล์ `Impact_Competitor_YYYYMMDD.txt` บน EAI S3 (`interface/in/ALM/` · โฟลเดอร์เดียวกับไฟล์ของ Job 2) — ● เปลี่ยนต้นทาง 2026-09-30 จากวิว `COMPETITOR_IMPACT_VIEW` (SQL Server) · 16 คอลัมน์ · วัดไฟล์จริง 4,109 แถว · `PERIOD_MONTH` เป็น เลขเดือน (8) ต่างจากไฟล์ของ Job 2 ที่เป็นวันที่เต็ม	อ่านทีละ 10,000 แถว · deduplicate ด้วยคีย์ (`impact_process_id`, `competitor_store_code`, `period_key`) ก่อน ยิงเข้า DB · แถวที่ `competitor_store_code` ว่างต้องแปลงเป็น NULL แล้ว dedup ด้วย `branch_th` แทน	ไม่ dedup ก่อน จะชน `uq_impact_competitor` แล้วทั้ง chunk fail
งวดนี้ Job 2 นำเข้าแล้วหรือยัง (● มติผู้ใช้ 2026-10-05)	`sgi_fgi_impact_processes` ของงวดที่ขอ (`impact_month`)	ไฟล์มีแถว แต่งวดนี้ไม่มีแถวแม่เลยสักแถว = Job 2 ยังไม่ได้นำเข้า (Job 2/3 trigger ด้วย S3 event แยกกัน ไฟล์คู่แข่งมาก่อนได้)	ล้ม `JOB3_PARENT_NOT_READY` · ไม่ insert · ไม่ย้ายไฟล์ ให้รันซ้ำหลัง Job 2 เสร็จ — ถ้าปล่อยตามกติการายแถวจะข้ามทุกแถว จบ SUCCESS แล้วไฟล์ถูกย้ายเข้า backup = ข้อมูลคู่แข่งทั้งงวดหายเงียบ · ร้านที่ขาดแถวแม่บางร้านยังข้ามรายแถว + alert ตามเดิม

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว ไม่งั้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
ขนาด chunk	10,000 แถว	พฤติกรรมเดิมของ `ImportImpactCompetitor`
ตัวกรองโซน	ไม่มี — ยึดระบบเดิม (มติผู้ใช้ 2026-10-05)	ระบบเดิมมีโค้ดกรองโซนแต่ comment ทิ้ง และ `zoneArr` ไม่เคยถูกเติมค่า
คีย์กันซ้ำ	`UNIQUE (impact_process_id, competitor_store_code, period_key)` — คีย์เป็นสาขา ไม่ใช่แบรนด์	`uq_impact_competitor` ใน DDL (แก้ 2026-09-09)

2.5 Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	0 07 7 * *	แก้ไขได้	ใช้สคริปต์ /appstore/SPS/FGI/schedule/FGI_ImportCompetitor.sh; Operations ตรวจ deployment path และ owner permission ก่อนขึ้น production
Argument (งวด)	2569|06	แก้ไขได้	รูปแบบ YYYY|MM
Chunk Size	10000	แก้ไขได้	จำนวนแถวต่อรอบ insert
Source View	COMPETITOR_IMPACT_VIEW	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	SELECT DISTINCT / map คอลัมน์ NAMT -> NAME_TH, BRANCHT -> BRANCH_TH

2.6 หลักฐานที่ต้องเก็บตอนรัน

Evidence	Job-specific value	Acceptance
Input identity	Period year/month and competitor impact data from ALLMAP Impact_Competitor_*.txt read from EAI S3 (interface/in/ALM/).	บันทึกไฟล์/คีย์ธุรกิจ/งวดที่รับเข้าใน run record
Output identity	FGI_IMPACT_COMPETITOR rows for the target period; run status is success/no-data/failed with inserted-count reconciliation.	กระทบยอด input · สำเร็จ · ตก · ข้าม

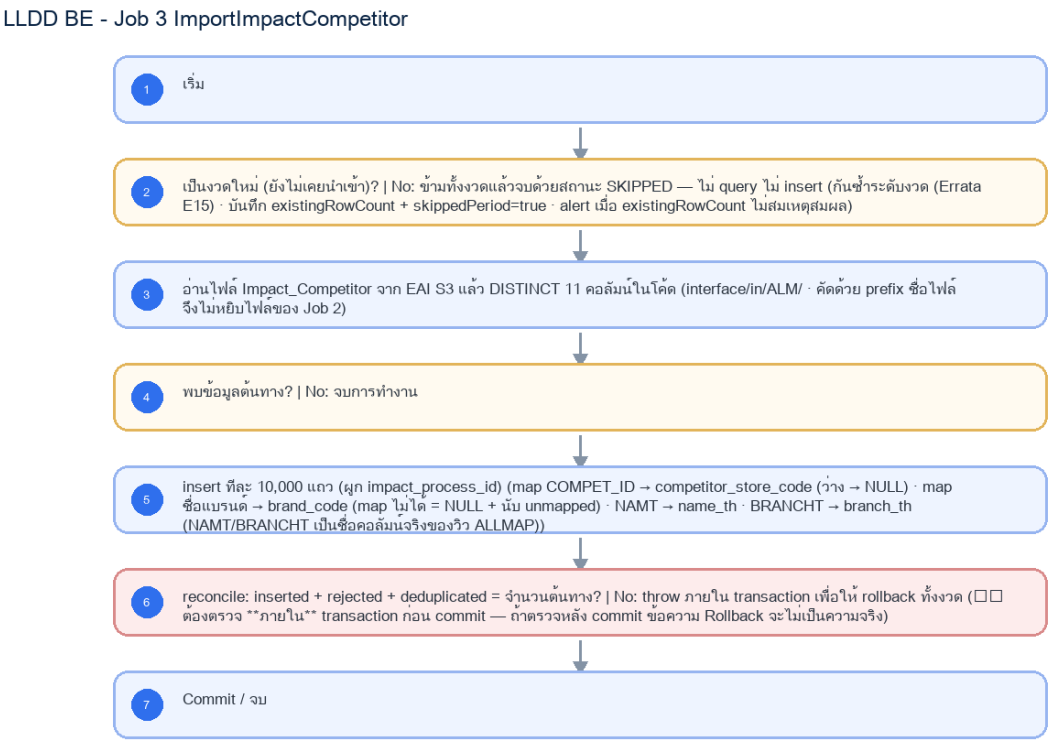
Evidence	Job-specific value	Acceptance
Dedup proof	UNIQUE(impact_process_id, competitor_store_code, period_key) — คีย์เป็นสาขา ไม่ใช่แบรนด์ (แก้ 2026-09-09); source row ซ้ำในวิวต้อง deduplicate ก่อน upsert · สัญญาระดับ job คือ period-level skip — ✓ ปิดแล้ว 2026-09-13 (ข้อ 2.23): คงพฤติกรรมเดิม (งวดมีข้อมูลแล้ว = ข้ามทั้งงวด) แต่ต้องเทียบ `existingRowCount` กับ `sourceCount` ทุกรอบแล้ว alert เมื่อของเดิมน้อยกว่าต้นทาง · หลักฐาน: ข้อมูลจริง 88 งวด (2017-2025) insert จบภายในวันเดียวทั้ง 88 งวด · งวดที่ insert หลายวัน = 0 จึงไม่เคยมีการนำเข้าไม่ครบแล้วเติมทีหลัง และไม่มีคำสั่ง `DELETE FROM FGI_IMPACT_COMPETITOR` ที่ใดเลยทั้ง repo	รันซ้ำด้วย fixture เดิมต้องไม่เกิดคีย์ธุรกิจซ้ำ
Transaction proof	validate งวดก่อนอ่าน; upsert ทีละ chunk และ commit หลัง reconcile จำนวน input/success/reject ของ chunk ตรงกัน	จำลอง error กลางทางแล้วไม่เหลือแถวค้างนอกขอบเขตที่ระบุ
Security proof	ALLMAP datasource ใช้ secretRef และ TLS verify-full; จำกัด DB user เป็น SELECT เฉพาะ source view	config/log/error ไม่มีความลับแบบข้อความธรรมดา

2.7 โค้ด Java เดิมที่อ้างอิง

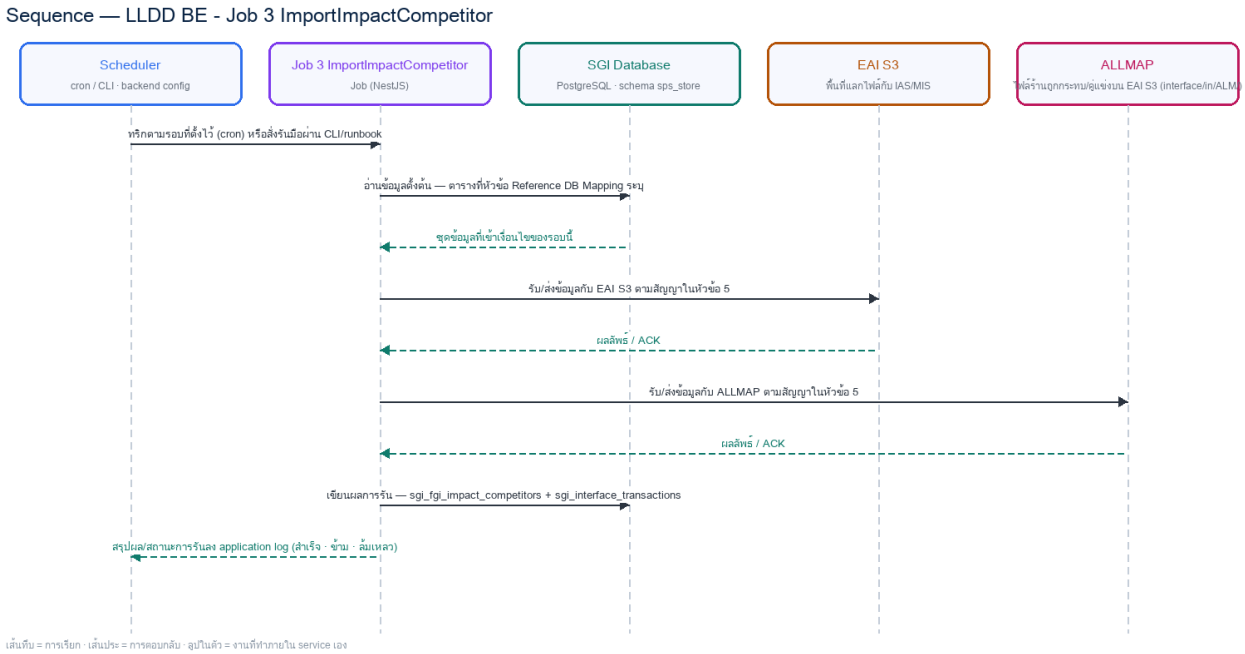
Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ImportImpactCompetitor.java	16-48	Legacy main entrypoint and notification wrapper.
fcsJar/src/th/co/gosoft/fgi/controller/ImportController.java	483-598	Validate params, skip duplicates, query source, chunk insert sgi_competitors.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportJdbc.java	200-241	Count existing period, query COMPETITOR_IMPACT_VIEW, insert FGI_IMPACT_COMPETITOR.

Line ranges refer to the legacy Java implementation under `batchjob/fcsJar/` (path นับจากราก `sbp-prototype/`). Use these ranges to preserve business behavior while implementing the target Node job.

3. Flow และ Sequence Diagram



รูปที่ 1: Implementation flow: LLDD BE - Job 3 ImportImpactCompetitor



รูปที่ 2: Sequence diagram: LLDD BE - Job 3 ImportImpactCompetitor

4. Reference DB Mapping

อ้างอิงสำหรับ implement job เท่านั้น ไม่ใช่งานออกแบบ/สร้างหน้า Database

Table / Object	R/W	Usage
sgi_fgi_impact_competitors	W	insert รายงวด (งวดล่าสุดต่อร้าน) ดึงจาก ALLMAP · ช่องทางต้นทาง ALM เก็บที่ sgi_fgi_impact_processes.datasource

5. โค้ดจริงอยู่ที่ไหน

เอกสารนี้ ไม่วาง skeleton code — job ทั้ง 11 ตัว implement แล้ว การมีโค้ดสองชุด (ในเอกสาร + ของจริง) ทำให้หลุดจากกันแน่นอน · อ่านสัญญาที่หัวข้อ 2 แล้วดูของจริงตามตารางนี้

สิ่งที่ต้องดู	ที่อยู่
ชื่อ job ที่ลงทะเบียน	`sgi-job-3` ใน `SBP/srm-sps-spsap-sop-sgi-batch/src/main.ts`
service ของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-3-*.service.ts`
เทสของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-3-*.spec.ts`
SQL ที่ใช้จริง	อยู่ในไฟล์ service เดียวกัน (repo นี้ไม่แยก `*.sql.ts`)
คำอธิบายภาษาคนของ job	batchjob/JOB-03-* อธิบายละเอียด

6. Acceptance Criteria

- พารามิเตอร์รับผ่าน `INPUT` (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ `@Cron` ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนพึ่ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (`BATCH_START`/`BATCH_END` + `runId` — `src/main.ts` ทำให้แล้ว) และบันทึกลง `integration_log` อัตโนมัติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

7. Unit Test Case

30 เคส · ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง (mock ที่ขอบ repository/client) · ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่ · ● เทสที่ mock query runner ต้องป้อนรูป tuple `[rows, rowCount]` สำหรับ UPDATE/DELETE (กฎ #140)

Argument / Input Validation

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-01	แปลง raw argument รูปแบบเดิม (`YEAR|MONTH`) เป็นรูปแบบใหม่ (named option)	`2026|07`	ได้ `year=2026, month=07`
UT-02	validate ไม่ผ่านเมื่อระบุ `year`/`month` ไม่ครบคู่ หรือ `month` นอกช่วง 1-12	เช่น `year=2026` ไม่ระบุ `month`, หรือ `month=13`	validate ไม่ผ่าน → error, ไม่เรียก Service
UT-03	เติม default งวดก่อนหน้าเมื่อไม่ระบุปี/เดือน (รวมกรณีข้ามปี)	ไม่ระบุ `year`/`month`; ทดสอบเวลาปัจจุบัน = 15 ส.ค. 2026 และ 10 ม.ค. 2026 (Asia/Bangkok)	ได้ `year=2026,month=7` และ `year=2025,month=12` ตามลำดับ
UT-04	ไม่ระบุ argument ใดๆเลยทั้งหมด	raw argument ว่าง (`[]`)	validate ผ่านตามปกติ (ไม่ใช่ error): เติม default งวดเป็นเดือนก่อนหน้า แล้วเรียก Service ต่อได้

การตรวจข้อมูลซ้ำในงวด

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-05	มีข้อมูลของงวดนี้อยู่แล้ว	count = 5	ไม่เรียก S3 เลย, ส่งผลลัพธ์กลับเป็น SUCCESS พร้อม note, ไม่เปิด transaction

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-06	ยังไม่มีข้อมูลของงวดนี้	count = 0	ไปทำขั้นตอนดึงไฟล์จาก S3 ต่อ

การดึงไฟล์จาก S3

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-07	มีหลายไฟล์ วันที่ในชื่อไฟล์ต่างกัน	ไฟล์ `..._20260916.txt`, `..._20260901.txt`	เลือกไฟล์ `20260916`
UT-08	วันที่ในชื่อไฟล์เท่ากัน แต่เวลาแก้ไขต่างกัน	ไฟล์ 2 ตัว `YYYYMMDD` เดียวกัน, `LastModified` ต่างกัน	เลือกไฟล์ที่ `LastModified` ใหม่ล่าสุด
UT-09	ต้อง list ไฟล์หลายหน้า	S3 คืนผลแบบแบ่งหน้า 2 หน้า	รวมไฟล์จากทุกหน้าก่อนเลือกไฟล์ล่าสุด
UT-10	ไม่มีไฟล์ตรง pattern เลย	ไฟล์ในถังไม่มีชื่อที่ตรง `Impact_Competitor_YYYYMMDD.txt`	สรุปผลเป็น `FAILED` ด้วย error ไม่พบไฟล์
UT-11	เรียก S3 แล้ว error	S3 ตอบ error ตอน list	สรุปผลเป็น `FAILED`

การแปลงข้อมูล (Parsing)

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-12	header ไม่ถูกต้อง	ไฟล์ที่บรรทัดแรกผิด format	สรุปผลเป็น `FAILED`, ไม่อ่าน row ข้อมูลต่อ
UT-13	stream ขาดกลางทาง	stream ที่หยุดกลางไฟล์	สรุปผลเป็น `FAILED`, ไม่คืนผลลัพธ์แบบครึ่งๆกลางๆ
UT-14	row มีเดือน/ปีผิดรูปแบบ	row ที่ `PERIOD_MONTH` ไม่ใช่ตัวเลข	row นั้นถูกเก็บเป็น rejected พร้อมเหตุผล, อ่าน row ถัดไปต่อ
UT-15	row มีวันที่เปิด/ปิดร้านคู่แข่งผิดรูปแบบ	row ที่ `OPEN_DATE` ผิด format	row นั้นถูก reject พร้อมเหตุผล ไม่แปลงเป็นค่าว่างเปล่าเงียบๆ
UT-16	row ถูกต้องครบ	row ที่ทุก field ถูกต้อง	เก็บเข้า accepted list พร้อม `data_source=ALM`, ใช้เดือน/ปีจากคอลัมน์ในไฟล์ (ไม่ใช่จาก argument)
UT-17	มีเครื่องหมาย delimiter อยู่ใน double quote	row ที่มี `"a|b"` ในคอลัมน์	ไม่ตัด column ผิดตำแหน่ง
UT-18	เดือน/ปีในไฟล์ต่างจาก `year/month` ที่ระบุมาตอนรัน batch	argument `year=2026, month=7` แต่ไฟล์มี `PERIOD_YEAR=2026, PERIOD_MONTH=8`	ไม่ filter row ออก ยังเก็บเข้า accepted list ตามเดือน/ปีในไฟล์ (คง behavior เดิม)

การบันทึกข้อมูล / Transaction

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-19	จำนวน record ไม่เกินขนาด batch สูงสุด	record 500 แถว	insert รอบเดียว, จำนวนที่ insert ตรงกับจำนวน record
UT-20	จำนวน record เกินขนาด batch สูงสุด	record 25,000 แถว (batch สูงสุด 10,000/รอบ)	แบ่ง insert เป็น 3 รอบ (10,000, 10,000, 5,000) ภายใน transaction เดียว
UT-21	insert สำเร็จทุก batch	ทุก batch insert ไม่มี error	commit transaction, จำนวนที่ insert สำเร็จเท่ากับจำนวน accepted record ทั้งหมด, สรุปผล SUCCESS
UT-22	insert ล้มเหลวที่ batch กลาง	batch ที่ 2 จาก 3 ล้มเหลว	throw error ออกจาก transaction, rollback ทั้งหมด (รวม batch ที่ insert ไปแล้วก่อนหน้าในรอบเดียวกัน), สรุปผลเป็น `FAILED`

กรณีไม่มีข้อมูล (No Data)

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-23	ทุก row ในไฟล์ผิดรูปแบบทั้งหมด	ทุก row ถูก reject	สรุปผลเป็น `NO_DATA`, `rejected_rows = total_rows`, `accepted_rows = 0`, ไม่เปิด transaction

ผลลัพธ์มาตรฐานและ Error

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-24	เกิด error กลางทาง (เช่น database error ระหว่าง transaction)	mock error หลังผ่านขั้นตอน S3/parse บางส่วนไปแล้ว	ผลลัพธ์ `FAILED` ยังคง `year`, `month`, `source_file`, `started_at`/`finished_at` และ counters ที่ทราบก่อนเกิด error ไว้ครบ
UT-25	error message ที่แสดงใน result ต้อง sanitize	error ที่มี connection string/credential อยู่ใน stack trace เดิม	`error.message`/`error.cause` ไม่มีข้อมูลอ่อนไหวหลุดออกมา

การแจ้งเตือน / Notification

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-26	หา template และส่งอีเมลสำเร็จ	mock ทั้งสองขั้นตอนสำเร็จ	อีเมลถูกส่งด้วยเนื้อความที่ render จากผลลัพธ์จริง
UT-27	หา template ไม่พบ	mock query template คืนค่าว่าง	log ไว้เท่านั้น ไม่ throw error, ไม่กระทบผลลัพธ์ที่ commit แล้ว
UT-28	ส่งอีเมลผ่าน SMTP ล้มเหลว	mock SMTP ล้มเหลว	log ไว้เท่านั้น, ผลลัพธ์ `SUCCESS`/`NO_DATA` เดิมไม่เปลี่ยนเป็น `FAILED`
UT-29	ผลลัพธ์เป็น `SUCCESS`/`NO_DATA`	ทำงานจบปกติ	exit code = `0`
UT-30	ผลลัพธ์เป็น `FAILED`	เกิด error ณ จุดใดก็ตาม	exit code = `1`

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ImportImpactCompetitor.java — lines 16-27

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ImportImpactCompetitor.java
Original lines	16-27
Responsibility	Legacy main entrypoint and notification wrapper.

```
public class ImportImpactCompetitor {

private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");
private static ImportController importController = (ImportController) context.getBean("importController");
private static FgiConfig config = (FgiConfig) context.getBean("fgiConfig");

public static void main(String[] args) {
EmailTemplateInformStatusBean informBean = new EmailTemplateInformStatusBean();
try{
LogUtils.info(ImportImpactCompetitor.class,"Start => import FGI_IMPACT_COMPETITOR");
informBean.setStartDateTime(Calendar.getInstance(Locale.US).getTime());
informBean.setJobName(FgiConstant.JOB_NAME_IMPORT_IMPACT_COMPETITOR);
```

J1. ImportImpactCompetitor.java — lines 37-48

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ImportImpactCompetitor.java
Original lines	37-48
Responsibility	Legacy main entrypoint and notification wrapper.

```
informBean.setErrorMsg(ex.toString());
informBean.setEndDateTime(Calendar.getInstance(Locale.US).getTime());

SendMailUtil.sendMailToAdmin(informBean, config, true);
} catch (Exception e){
LogUtils.error(ImportImpactCompetitor.class,e);

}
}
LogUtils.info(ImportImpactCompetitor.class,"End => import FGI_IMPACT_COMPETITOR ");
}
}
```

J2. ImportController.java — lines 483-494

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ImportController.java
Original lines	483-494
Responsibility	Validate params, skip duplicates, query source, chunk insert sgi_competitors.

```
public void importImpactCompetitor(String[] params,EmailTemplateInformStatusBean informBean) throws Exception{
String month = new String();
String year = new String();
String[] zoneArr = {};
String[] paramsArr = {};
boolean flagSendMail = true;

if(params.length > 0){

paramsArr = params[0].split("\\|",-1);

if(paramsArr.length == 2 && !StringUtils.isEmpty(paramsArr[0]) && !StringUtils.isEmpty(paramsArr[1])){
```

J2. ImportController.java — lines 587-598

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ImportController.java
Original lines	587-598
Responsibility	Validate params, skip duplicates, query source, chunk insert sgi_competitors.

```
});
if(totalRecordInsert == impactCompetitorList.size() && flagSendMail){ // insert == select
informBean.setStatus(FgiConstant.JOB_STATUS_SUCCESS);
informBean.setEndDateTime(Calendar.getInstance(Locale.US).getTime());
SendMailUtil.sendMailToAdmin(informBean, config, true);
}else if(totalRecordInsert != impactCompetitorList.size() && flagSendMail){
informBean.setStatus(FgiConstant.JOB_STATUS_FAIL);
informBean.setNote("totalRecordInsert != impactCompetitorList.size()");
informBean.setEndDateTime(Calendar.getInstance(Locale.US).getTime());
SendMailUtil.sendMailToAdmin(informBean, config, true);
}
}
```

J3. ImportJdbc.java — lines 200-211

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportJdbc.java
Original lines	200-211
Responsibility	Count existing period, query COMPETITOR_IMPACT_VIEW, insert FGI_IMPACT_COMPETITOR.

```
public int countRowCompetitor(String year, String month,String[] zoneArr){
ArrayList<String> params = new ArrayList<String>();
String zoneStr = new String();
params.add(year);
params.add(month);
String sql = "select count(1) count_rec from fgi_impact_competitor where year = ? and month = ? ";

/*if(zoneArr != null && zoneArr.length > 0){
zoneStr = TextUtils.arrayElementToStringHasQuote(zoneArr);
sql += " AND ZONE_CODE in ("+zoneStr+")";
}*/
```

J3. ImportJdbc.java — lines 230-241

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportJdbc.java
Original lines	230-241
Responsibility	Count existing period, query COMPETITOR_IMPACT_VIEW, insert FGI_IMPACT_COMPETITOR.

```
return ImpactCompetitorList;
}

public int[] insertImpactCompetitor(List<ImpactCompetitorBean> impactCompetitorList){
StringBuffer sql = new StringBuffer();
sql.append(" INSERT INTO FGI_IMPACT_COMPETITOR");
sql.append(" (STORECODE_I,COMPET_ID,NAME_TH,NAME,BRANCH_TH,ZONE_CODE,SUBZONE_CODE,OPEN_DATE,CLOSE_DATE,MONTH,YEAR,DATAS
OURCE,IMPACT_COMPETITOR_ID)");
sql.append(" VALUES(?,?,?,?,?,?,?,?,?,?,?,?,SEQ_FGI_IMPACT_COMPETITOR.NEXTVAL)");
int[] result = jdbcTemplate.batchUpdate(sql.toString(),new ImpactCompetitorBatch(impactCompetitorList));
LogUtils.info(getClass(),"insert FGI_IMPACT_COMPETITOR : " + result.length + " Record (s)");
return result;
}
```